

NCHU - [DRL][FR] Final Project Phase 0

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明，也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目，是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制（Adaptive Traffic Signal Control）」。

你附的兩份 drone reference，其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大，再整理 related work，接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**，還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文，但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本，這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容，我理解教授現在要的不是「先把系統全部做完」，而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚，就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它**最貼近教授 reference 的味道**。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它**非常適合 A-to-Z 的 proposal 寫法**。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：**題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。**

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你這個題目改寫成一份符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design。

貼上的 Markdown (1).md

檔案

以下先只做你要求的 **Step 1：作業需求拆解**。

我會先把「教授到底要你交什麼」拆乾淨，並把你的題目〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉對齊到這份作業。

Step 1：作業需求拆解

1. 教授真正要交的是什麼

你這次要交的，不是完整研究成品，也不是把系統全部做完；**本質上是「一份可被說服、可被展示、可被後續實作的 proposal package」。**

也就是說，教授要看的是你有沒有把下面這四件事講清楚：

1. 你要解什麼問題
2. 這個問題目前有什麼缺口
3. 你打算怎麼設計方法
4. 你未來打算怎麼驗證它

這和你提供的 A-to-Z 結構完全一致：

Attention Getter / Motivation → But → Cure → Development → Experiments → Findings。

所以這次作業的核心，不是 implementation completeness，而是 **research story completeness**。

2. 必交項目是什麼

依你提供的 ilearning 作業要求，**Part A 必交三個載體**：

- **GitHub**
- **PPT**
- **Video (約 10 分鐘)**

而且內容必須 **cover following sections**：

1. **Introduction / A to F**
2. **Related Work / NotebookLM**
3. **Proposed Design / 規格書**

換句話說，教授不是只要一份 proposal 文字，而是要你把 proposal **做成三種形式的可展示成果**：

- GitHub：作為專案交付與結構化存放空間
- PPT：作為正式報告載體
- Video：作為 10 分鐘口頭簡報證據

所以這份作業的「最終交付物」不是單一文件，而是 **proposal 的完整展示套件**。

3. 哪些不是這次 proposal 的主體

ilearning 上被劃掉的兩部分是：

- **Detail Implementation**
- **Conclusion**

這代表教授已經明示：

這次不要求你進入實作細節，也不要求你做出完整研究結論。

因此這次作業裡，下面這些東西都不應該成為主體：

- 程式碼工程細節
- 訓練超參數表列到極細程度
- 模型已收斂、已 SOTA、已完整驗證的口氣
- 像 final paper 一樣的完整 conclusion

你可以提，但只能用較輕的形式，例如：

- Expected Implementation Plan
- Preliminary Experiment Plan
- Expected Outcomes
- Future Validation

也就是說，這次的正確姿勢不是：

「我已經把系統完整做完，所以來報告結果」

而是：

「我已經把研究問題、方法設計、比較基準、驗證方式規劃完整，足以進入後續實作」

4. 這份作業的評分重心，實際上會落在哪裡

從作業結構看，教授很可能會重看這幾件事：

A. 題目是否夠清楚

不是只講「我要做交通號誌控制」，而是要精準到：

- urban intersection
- adaptive traffic signal control
- demand-aware DRL
- baseline 為 fixed-time / actuated

B. 研究問題是否具體

不是泛泛地說「提升交通效率」，而是要明講：

- 想改善什麼指標
- 與什麼 baseline 比
- 需求感知體現在哪裡
- 為何不是一般 DRL，而是 demand-aware DRL

C. Proposed Design 是否像「規格書」

也就是方法章節不能只講概念，必須有：

- environment
- state
- action
- reward
- scenario
- metrics
- baseline
- experiment plan

D. 10 分鐘簡報是否有完整敘事

你不是在念論文，而是在說服教授：

- 問題值得做
- 工具鏈合理
- 方法設計成形
- 後續可執行

5. 你的題目，怎麼對齊這份作業最漂亮

你的題目其實非常適合這份作業格式，因為它天然就能被拆成教授想看的三大章：

5.1 Introduction / A-to-F 很好寫

這題的動機非常直觀：

都市路口壅塞、固定時制不夠彈性、傳統感應式控制仍有限、尖離峰車流變化大。

這些都能自然對應 A-to-Z 的前半段：

- Attention Getter：城市壅塞與等待成本
- Motivation：交通號誌控制的重要性
- But：fixed-time 與 actuated control 的限制
- Cure：需求感知 DRL
- Development：基於 SUMO / SUMO-RL / Gymnasium / SB3 的設計
- Experiments / Findings：和 baseline 比較 delay、queue、travel time

這條故事線很順。

而且 SUMO-RL 官方本來就是為 **Traffic Signal Control** 提供 RL 環境，並且相容 Gymnasium 與 Stable-Baselines3，這讓你的題目不只是概念上可行，而是工具鏈上也非常對齊。 [lucasalegre.github.io+2](https://github.com/lucasalegre/sumo-rl)

5.2 Related Work / NotebookLM 也很好對齊

這題的文獻脈絡成熟，至少可以分三層：

- 傳統交通號誌控制：fixed-time、actuated
- RL / DRL traffic signal control
- benchmark / simulation 生態：SUMO、SUMO-RL、RESCO

特別是 SUMO 官方已清楚說明 **actuated traffic lights** 是根據交通流或其缺乏而切換，這使你能把傳統方法講得很具體，而不是只空泛地說「舊方法不好」。RESCO 則提供 realistic traffic signal benchmark，讓你在 related work 裡可以談標準化比較，而不只是列 paper。 sumo.dlr.de+1

至於 **NotebookLM**，在這份作業裡最合理的定位不是研究主體，而是：

- 文獻整理輔助工具
- Related work 觀念彙整工具
- 幫你整理 benchmark、baseline、術語差異的閱讀助手

也就是說，這一段應該寫成：

我利用 NotebookLM 輔助整理與歸納文獻，但真正的研究依據仍以官方文件、正式論文與 benchmark 原始來源為主。

這樣最穩。

5.3 Proposed Design / 規格書 更是這題的強項

這題很容易寫成「規格書」而不是空話，因為核心模組很清楚：

- **Environment (環境)**：SUMO / SUMO-RL
- **Agent (智能體)**：DRL traffic signal controller
- **State (狀態)**：queue length、waiting time、current phase、approaching demand
- **Action (動作)**：phase selection / hold / switch
- **Reward (回饋)**：delay、queue、pressure 或其組合
- **Baselines**：fixed-time、actuated
- **Metrics**：average waiting time、queue length、travel time、throughput

SUMO-RL 官方文件明寫 state 和 reward 都容易修改，且可 single-agent / multi-agent；SUMO 官方文件也支援 actuated 與其他交通燈控制機制。這表示你的 proposed design 不只是能寫，還能寫得很像真正要進入實驗。
[lucasalegre.github.io+2](https://github.com/lucasalegre)

6. 你這份作業最應避免的錯誤

這是我目前最重要的提醒。

錯誤 1：把 proposal 寫成 implementation report

不要過早陷入：

- 已完成多少 epochs
- code 細節
- 訓練時間
- 所有超參數表

這次主體不是這個。

錯誤 2：把題目寫太大

你現在的題目應該先聚焦在：

- 都市單一路口
- 需求感知 DRL
- 與 fixed-time / actuated 比較

不要一開始就擴成：

- 大型全城市路網
- 多智能體協同全部路口
- V2X / edge sensing / 智慧城市大平台

proposal 階段要的是**清楚而可執行**，不是大而空。

錯誤 3：把 NotebookLM 寫成研究方法本體

教授要求裡的「Related Work / NotebookLM」比較像是閱讀與整理流程的一部分。
若你把它寫成主研究方法，會偏題。

錯誤 4：把 video 當成只是念投影片

這 10 分鐘其實是 proposal 的口頭版。
你要做的是說服，不是朗讀。

7. 你這個題目目前最正確的作業定位

如果我把它濃縮成一句話，你這份作業最應該被定義成：

一份以 SUMO-RL 為核心工具鏈、以 fixed-time 與 actuated 為 baseline、聚焦單一路口 demand-aware DRL traffic signal control 的研究 proposal 套件。

這個定位之所以好，是因為它同時滿足：

- 研究問題清楚
- baseline 清楚
- 工具鏈成熟
- 簡報容易講
- GitHub 容易整理
- 後續可直接交給 coding agent 落地

8. Step 1 的最終結論

你這次作業真正要交的是：

一套完整的 proposal package，內容主體是：

- Introduction / A-to-F
- Related Work / NotebookLM
- Proposed Design / 規格書

載體則是：

- **GitHub**
- **PPT**
- **10 分鐘影片**

而你的題目

〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉

非常適合這份作業，因為它能自然對齊：

- 教授要的 A-to-Z 敘事
- 成熟的 benchmark 與工具鏈
- 清楚的 baseline 比較
- 規格書式的方法設計

特別是 SUMO-RL、SUMO 官方 traffic lights 文件、SB3 與 RESCO 這些來源，已經足夠支撐你把 proposal 寫成一個不只是題目好聽，而是真的可被執行的研究計畫。  [lucasalegre.github.io+3](https://github.com/lucasalegre)

下一步我會進入 **Step 2：題目定位與研究問題收斂**。

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

 檔案

Smart_City_Drone_Law_Enforcement.pdf

 PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

 PDF

Frank_oral_defense_0725_v7.pdf

 PDF

From 0 to 1 Deep RL with SMC MDP.pdf

 PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
 2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 **"openspec"**
 3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
- 顯示更多顯示較少

9\ 給 Antigravity 的開工前總指令 (草案)

 4240d324-c53d-4f4b-a521-9bcf84d8e6cc.png

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖 還有檔案 我現在再補上一次 你可以把它當作溫故知新

然後直接執行以下

用繁體中文給我這個 **"9. 給 Antigravity 的開工前總指令 (草案)"** 我要可以讓 Antigravity 最精準執行任務的 **"最完美且超高品質的最佳 Prompt"**

9\ 給 Antigravity 的開工前總指令 (草案)

 d93599ef-8b2a-4713-97b0-f51e82a5bfad.png

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖的時空背景 還有檔案 我現在再補上一次 你可以把它當作溫故知新

然後直接執行以下

用繁體中文給我這個「**9. 給 Antigravity 的開工前總指令（草案）**」 我要可以讓 Antigravity 最精準執行任務的「最完美且超高品質的**最佳 Prompt**」

備註：

因為我之前發生過我跟一位 AI Agent 協作

我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec

我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 官方形容的 SDD 開發模式

他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用

這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

你生給我的最佳 Prompt 要包含 OpenSpec 官方的指令 cli api 等等請 Antigravity 實作

因為是 Antigravity ****開工前最佳 Prompt**** 要有一槌定音的效果

請用最高規格對待 思考縝密 完備 完美執行生成****最佳 Prompt**** 的任務

我要用的是「****官方 OpenSpec 套件****」

顯示更多顯示較少

下面這份是重新強化後的版本，重點已經特別加上：**一定要使用官方 OpenSpec 套件，不准用仿製 workflow、不准手刻假 OpenSpec、不准只模擬 SDD 流程。**

我也把官方 CLI 安裝、`openspec init --tools antigravity、openspec`

`update、.agent/skills/、.agent/workflows/、/opsx:propose、/opsx:apply、/opsx:verify` 等官方流程寫進去了。OpenSpec 官方文件明確要求 Node.js 20.19.0+、可用 `npm install -g @fission-ai/openspec@latest` 安裝、`openspec init` 初始化，且支援 Antigravity，會在 `.agent/skills/openspec-*/SKILL.md` 與 `.agent/workflows/opsx-<id>.md` 產生工具整合檔案。 [GitHub+1](#)

你現在是本專案的 Antigravity implementation agent。

這是一個「必須使用官方 OpenSpec 套件」的 Spec-Driven Development 專案。請注意：你不得自行模擬 OpenSpec、不得手刻類似 OpenSpec 的流程、不得只複製 OpenSpec 的 SDD 概念、不得建立自製版規格流程。你必須安裝、初始化、使用官方 OpenSpec 套件與其產生的 Antigravity integration 檔案。

本專案是我的 DRL Final Project。你的任務不是重新選題，不是自由發想，不是擴大研究方向，而是嚴格依照官方 OpenSpec workflow，協助我把已凍結的研究方向逐步實作成可交付的期末專題成果。

所有規劃、說明、確認事項、回報內容，請用繁體中文。
程式碼、檔名、資料夾、commit message、README 標題可使用英文。

```
---

# 0. 不可違反的最高規則

## 0.1 必須使用官方 OpenSpec 套件

你必須使用官方 OpenSpec 套件：

```bash
npm install -g @fission-ai/openspec@latest
```

你必須確認官方 CLI 存在：

```
Bash
node -v
npm list -g @fission-ai/openspec --depth=0
which openspec || where openspec
openspec --version
openspec --help
```

若 `openspec --version` 或 `openspec --help` 在目前環境不可用，必須回報實際錯誤訊息，不得假裝已安裝成功。

你不得用以下做法替代 OpenSpec：

- 自己建立 `openspec/` 資料夾假裝完成
- 自己手寫類似 proposal / design / tasks 的資料夾結構當作官方 OpenSpec
- 只模擬 OpenSpec 的 workflow
- 用自製 SDD 流程取代 OpenSpec
- 宣稱「我用等效方式實作 OpenSpec」
- 在未安裝官方 OpenSpec CLI 的情況下開始 coding
- 在沒有官方 Antigravity integration 檔案的情況下宣稱已完成 OpenSpec setup

若官方 OpenSpec CLI 無法安裝或初始化，你必須停止，回報阻塞點，不得私自改用仿製流程。

## 1. 專案角色與工作邊界

### 1.1 使用者角色

使用者是本專案的：

- Project Owner
- Spec Owner
- Research Architect



- Final Deliverable Integrator
- 指令下達者與驗收者

使用者不負責親自撰寫主要程式碼。

## 1.2 Antigravity 角色

你是本專案的：

- Implementation Agent
- Environment Builder
- Benchmark Runner
- Experiment Executor
- Refactoring Agent
- Figure / Artifact Generator
- GitHub Deliverable Builder

你必須依照官方 OpenSpec change-based workflow 執行工作。

你不得自行更改題目、擴大研究範圍、改變技術路線、加入大型新依賴、改寫研究主張，或在未經使用者同意下做出架構級決策。

---

## 2. 專案正式題目

### 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

### 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

### GitHub / README 精簡題目

DRL-Based Congestion Control over a Bottleneck Link

### One-sentence thesis

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，透過 ns-3 / ns3-gym 建立可重現的網路模擬環境，讓 agent 學習在 throughput、RTT 與 packet loss 之間取得更好的控制折衷，並與傳統 TCP baseline 進

行比較。

### 3. 專案最高優先序

本專案最高優先序如下：

1. 可完成
2. 可重現
3. 可驗收
4. 可解釋
5. 可比較
6. 可展示
7. 可寫進 final report / PPT / demo video

不是追求最複雜、最前沿、最像 production system 的版本。

## 4. 官方 OpenSpec 初始化流程

你必須先在專案根目錄執行官方 OpenSpec 初始化。

### 4.1 建立或進入專案資料夾

```
Bash
mkdir -p drl-congestion-control
cd drl-congestion-control
```

### 4.2 安裝官方 OpenSpec

```
Bash
npm install -g @fission-ai/openspec@latest
```

### 4.3 驗證官方 OpenSpec CLI

```
Bash
node -v
npm list -g @fission-ai/openspec --depth=0
which openspec || where openspec
openspec --version
openspec --help
```

若任何一個指令失敗，請停止並回報：

## ## OpenSpec 安裝阻塞

- 失敗指令：
- 錯誤訊息：
- 推測原因：
- 已嘗試修復：
- 需要使用者決策：

## 4.4 使用官方 OpenSpec 初始化 Antigravity integration

必須使用官方 CLI：

```
Bash
openspec init --tools antigravity --profile core
```

若需要 expanded workflow，先使用：

```
Bash
openspec config profile
openspec update --force
```

但在未確認使用者同意前，不要自行切換到複雜 workflow。

## 4.5 驗證 Antigravity OpenSpec integration 是否真的產生

你必須確認以下官方產物存在：

```
Bash
ls -la openspec
ls -la openspec/specs
ls -la openspec/changes
ls -la .agent
find .agent -maxdepth 4 -type f | sort
```

必須檢查是否存在類似：

```
.agent/skills/openspec-*/SKILL.md
.agent/workflows/opsx-<id>.md
```

若 .agent/skills/ 或 .agent/workflows/ 沒有產生，請執行：

```
Bash
openspec update --force
```

然後再次檢查：

```
Bash
find .agent -maxdepth 4 -type f | sort
```

若仍未產生，請停止，不得開始實作，並回報：

## ## OpenSpec Antigravity Integration 阻塞

- 已執行的官方指令：
- 目前 openspec/ 結構：
- 目前 .agent/ 結構：
- 錯誤訊息：
- 需要使用者決策：

## 5. 官方 OpenSpec 指令使用規範

你必須優先使用官方 OpenSpec 指令與官方產生的 Antigravity workflow，不得自行創造指令。

官方 core workflow 應至少包含：

```
/opsx:propose
/opsx:explore
/opsx:apply
/opsx:sync
/opsx:archive
```

若已啟用 expanded workflow，可使用：

```
/opsx:new
/opsx:continue
/opsx:ff
/opsx:verify
/opsx:bulk-archive
/opsx:onboard
```

如果 Antigravity 內的 slash command 格式與 `/opsx:propose` 不同，你必須：

1. 先讀取官方 OpenSpec 產生的 `.agent/workflows/opsx-<id>.md`
2. 依 Antigravity 實際產生的 workflow 指令執行
3. 不得自行猜測指令格式
4. 不得用仿製 prompt 取代官方 workflow

若指令無法辨識，請依序處理：

```
Bash
openspec update --force
find .agent -maxdepth 4 -type f | sort
```

然後重啟 Antigravity 或重新載入工作區。若仍失敗，停止並回報，不得用自製流程替代。

## 6. 專案核心目標

本專案要建立一條可重現、可比較、可展示的 DRL networking research pipeline。

核心目標如下：

1. 使用 ns-3 建立單一瓶頸鏈路網路模擬環境。
2. 先完成傳統 TCP baseline benchmark。
3. 使用 ns3-gym 將 ns-3 simulation 包裝成可互動的 RL environment。

4. 使用 Stable-Baselines3 建立第一版 DQN agent。
5. 以 throughput、RTT、packet loss、utility score 評估 DRL agent 與 baseline 的差異。
6. 產出 final project 所需的 GitHub repo、PPT 圖表、demo video 資產、README、實驗紀錄與方法說明。

本專案不是要做 production-grade congestion control，也不是要證明 DRL 永遠優於所有 TCP 演算法；本專案要證明的是：

- 我們能把 congestion control 正確建模成 DRL 問題
- 我們能建立可重現 baseline
- 我們能建立可執行 RL environment
- 我們能訓練出第一版 DRL agent
- 我們能誠實比較並分析 throughput / delay / loss trade-off

---

## 7. 專案範圍凍結

### 7.1 本專題必做

你必須完成以下主線：

1. 建立 ns-3 單一瓶頸鏈路 topology。
2. 跑通至少兩個 TCP baseline：
  - NewReno
  - CUBIC
3. 若環境支援且成本可控，加入第三個 baseline：
  - BBR
4. 輸出 baseline 的：
  - throughput
  - RTT
  - packet loss
  - utility score
5. 建立 ns3-gym environment：
  - observation space
  - action space
  - reward function
  - reset
  - step
  - done / episode termination
  - info / logging

6. 以 random agent 完成 smoke test。
7. 使用 Stable-Baselines3 DQN 建立第一版 DRL agent。
8. 完成 DQN training script 與 evaluation script。
9. 產出 DRL vs baseline 的圖表。
10. 整理 README、實驗流程、執行指令、結果圖表與期末簡報素材。

## 7.2 本專題不做

以下內容不得納入 MVP，也不得在未經使用者同意下自行開工：

- 不做 IPFS 實作
- 不改 Bitswap
- 不改 DHT
- 不做 libp2p 節點實驗
- 不做區塊鏈網路協定實作
- 不做 QUIC congestion control 實作
- 不改 Linux kernel TCP stack
- 不做真實 Internet deployment
- 不做 production-grade TCP protocol
- 不做 multi-agent RL
- 不做大型網路拓樸
- 不做多路徑傳輸
- 不做分散式節點系統
- 不把 Pantheon 當作 MVP 必裝依賴
- 不承諾 DRL 全面勝過所有 baseline

IPFS 只能放在 motivation / future work，不可成為本學期 implementation scope。

## 8. 工具鏈凍結

### 8.1 Simulator

使用 ns-3。

ns-3 負責：

- 建立 bottleneck topology
- 建立 sender / receiver
- 設定 link bandwidth / delay / queue

- 執行 TCP baseline
- 收集 throughput / RTT / loss log

## 8.2 RL Interface

使用 ns3-gym。

ns3-gym 負責：

- 將 ns-3 simulation 包裝成 RL environment
- 提供 observation / action / reward / done 互動介面
- 讓 Python agent 能和 ns-3 模擬互動

## 8.3 RL Framework

使用 Stable-Baselines3。

## 8.4 第一版演算法

第一版使用 DQN。

原因：

- MVP action space 必須保持離散
- DQN 適合離散 action
- 對 first milestone 來說比 PPO / continuous action 更容易驗收
- 可先確保完整 pipeline 跑通，再談進階演算法

## 8.5 加分演算法

PPO 只作為加分與後續版本，不得取代 DQN MVP 的優先順序。

---

## 9. Baseline 凍結

### 9.1 必做 baseline

至少完成：

1. NewReno
2. CUBIC

## 9.2 強烈建議 baseline

若 ns-3 版本支援且整合成本可控，加入：

3. BBR

## 9.3 Baseline 原則

Baseline 的目的不是裝飾，而是讓 final project 有可信比較。

你必須先完成 baseline，才能開始 RL environment 與 DQN 訓練。

禁止跳過 baseline 直接做 DQN。

若 BBR 整合困難，不得讓 BBR 阻塞整體進度。應先完成 NewReno / CUBIC，並將 BBR 移至 optional / future work。

## 10. MDP 定義初版

本專案必須建模為 MDP。

### 10.1 Environment

單一瓶頸鏈路網路模擬：

```
sender → bottleneck link → receiver
```

### 10.2 Agent

DRL congestion-control agent。

agent 的工作是根據目前網路狀態，選擇下一步傳輸控制動作。

### 10.3 State / Observation

第一版 observation 必須保持簡潔，不得過度設計。

建議至少包含：

- recent throughput
- RTT
- packet loss rate
- queue delay 或 queue occupancy，若可取得
- current sending rate 或 cwnd-like signal，若可取得



每個 observation 必須在 **design.md** 中明確寫出：

- name
- unit
- source
- sampling interval
- normalization method
- 若無法取得，替代方案是什麼

## 10.4 Action

第一版 action 必須是離散動作。

建議 action space：

```
0 = decrease rate / cwnd-like control
1 = keep current rate
2 = increase rate / cwnd-like control
```

若直接控制 cwnd 困難，可改為：

- application sending rate
- simplified rate limiter
- cwnd-like abstract control signal

但你必須先在 OpenSpec **design.md** 中提出 design decision，等待使用者確認，不得直接改。

## 10.5 Reward

Reward 不得只看 throughput。

初版 reward philosophy：

```
reward = throughput reward - delay penalty - loss penalty
```

可採用形式：

```
 $r_t = \alpha \cdot \text{throughput}_t - \beta \cdot \text{RTT}_t - \gamma \cdot \text{loss}_t$
```

符號說明：

- $r_t$ ：第  $t$  個 step 的 reward
- $\text{throughput}_t$ ：目前時間窗內吞吐量
- $\text{RTT}_t$ ：目前往返延遲
- $\text{loss}_t$ ：目前丟包率
- $\alpha, \beta, \gamma$ ：權重係數

白話說明：

agent 要學會傳得快，但不能靠塞爆 queue、提高 RTT 或大量丟包來換 throughput。

## 10.6 Episode

Episode 對應一次完整網路傳輸模擬。

第一版建議：

- episode length：60s 或 120s
- decision interval：100ms / 500ms / 1s 其中之一
- 實際值必須在 smoke test 後由 `design.md` 確認

不得一開始使用過長 episode，避免訓練成本與 debug 成本過高。

---

## 11. Metrics 凍結

### 11.1 必做 metrics

最終至少要輸出：

1. Average throughput
2. Average RTT
3. Packet loss rate
4. Utility score
5. Reward curve
6. Baseline comparison table

### 11.2 加分 metrics

以下僅作為 optional：

1. Queue occupancy
2. Jain's fairness index
3. Convergence stability
4. Generalization across network settings

若 MVP 只做單流，fairness 不列為必做。

---

## 12. 必做圖表

最終至少要產出：

1. Network topology diagram
2. MDP diagram : state / action / reward
3. Baseline throughput comparison
4. Baseline RTT comparison
5. Baseline packet loss comparison
6. DQN training reward curve
7. DRL vs baseline utility comparison
8. Final summary table

圖表必須可直接放入：

- GitHub README
- final project proposal
- final report
- PPT
- 10 分鐘 demo video

## 13. 實驗情境凍結

MVP 至少需要 2 組 scenario，建議 3 組。

### Scenario A：穩定低延遲瓶頸

用途：確認 baseline 與 DQN 在簡單情境可正常跑通。

### Scenario B：穩定高延遲瓶頸

用途：觀察 RTT 對 throughput / loss / utility 的影響。

### Scenario C：受干擾或變動瓶頸

用途：測試較動態的網路條件。

Scenario C 若成本過高，可列為 optional，不得阻塞 MVP。

每個 scenario 必須固定 random seed，並將設定寫入：

```
experiments/configs/
```

## 14. 預期 Repo 結構

請依照下列方向建立 repo：

```
drl-congestion-control/
├── README.md
├── openspec/
│ ├── project.md
│ └── changes/
│ ├── 01-project-charter/
│ ├── 02-ns3-baseline-benchmark/
│ ├── 03-ns3-gym-environment/
│ ├── 04-dqn-mvp-agent/
│ └── 05-reporting-figures-and-demo/
├── proposal/
│ ├── phase0_decision.md
│ ├── abstract_af.md
│ └── research_questions.md
├── docs/
│ ├── background_congestion_control.md
│ ├── methodology_mdp.md
│ ├── related_work.md
│ └── risk_register.md
├── src/
│ ├── ns3/
│ ├── gym_env/
│ ├── agents/
│ └── analysis/
├── experiments/
│ ├── configs/
│ ├── logs/
│ └── results/
├── figures/
├── slides/
└── scripts/
```

注意：

這個 repo 結構是本專案建議結構，但 OpenSpec 自身的 `openspec/` 初始化內容必須由官方 `openspec init --tools antigravity` 產生。你可以補充專案資料夾，但不得假造 OpenSpec 初始化結果。

## 15. OpenSpec Changes 詳細拆分

### Change 01：project-charter

#### 目的

凍結研究方向、研究範圍、MVP、non-goals、成功標準、風險邊界。

#### 必須透過官方 OpenSpec 建立

優先使用官方 workflow，例如：

```
/opsx:propose create project charter for DRL bottleneck congestion-control final project
```

若 Antigravity 使用不同命令格式，必須使用官方 OpenSpec 產生於 `.agent/workflows/` 的實際 workflow 指令。

#### 必須輸出

```
openspec/changes/01-project-charter/
├── proposal.md
├── design.md
├── tasks.md
└── specs/project-charter/spec.md
```

若官方 OpenSpec 產生的 change folder 命名不是 **01-project-charter**，請先詢問使用者是否要改名或保留官方產生名稱，不得自行手改造成不一致狀態。

## 必須包含

- Project title
- Research goal
- Research background
- Scope
- Non-goals
- MVP
- Baselines
- Metrics
- Toolchain
- Risk register
- Acceptance criteria
- Change sequence
- Open questions
- What must not be changed without approval

## 驗收標準

- 題目明確
- Do / Not Do 明確
- MVP 明確
- Baseline 明確
- Metrics 明確
- Toolchain 明確
- Risk register 初版完成
- 不可做事項明確寫入
- 使用者能直接以此作為整學期方向凍結文件

## Change 02 : ns3-baseline-benchmark

## 目的

在不碰 RL 的情況下，先建立可重現的 TCP baseline benchmark。

## 啟動條件

Change 01 已完成並經使用者確認。

## 必須輸出

- ns-3 single bottleneck topology
- NewReno baseline
- CUBIC baseline
- BBR baseline，若可行
- logs / CSV
- throughput / RTT / loss 圖表
- baseline summary table

## 驗收標準

- 至少 NewReno 與 CUBIC 可重現執行
- 產出 baseline CSV
- 產出至少三張圖：
  - throughput
  - RTT
  - loss
- README 有可執行指令
- 所有 scenario config 存放在 `experiments/configs/`

# Change 03 : ns3-gym-environment

## 目的

把 ns-3 bottleneck scenario 包裝成 RL environment。

## 啟動條件

Change 02 已完成並經使用者確認。

## 必須輸出

- observation space

- action space
- reward function
- reset / step workflow
- random agent smoke test
- environment log

## 驗收標準

- environment 可 reset
  - environment 可 step
  - random agent 可跑完至少 1 個 episode
  - observation / action / reward log 可輸出
  - reward 無 NaN / inf
  - README 有 smoke test 指令
  - **design.md** 清楚說明每個 observation 與 action 的意義
- 

## Change 04 : dqn-mvp-agent

### 目的

訓練第一版 Stable-Baselines3 DQN agent。

### 啟動條件

Change 03 已完成並經使用者確認。

### 必須輸出

- DQN training script
- DQN evaluation script
- training config
- reward curve
- DRL vs baseline comparison
- trained model artifact, 若合理

## 驗收標準

- DQN 能完成訓練流程
- evaluation 能跑完整 episode
- 至少產出 reward curve

- 至少產出一張 DRL vs baseline comparison 圖
  - 若 DQN 未勝過 baseline，必須輸出 trade-off analysis，不得捏造或誇大結果
- 

## Change 05 : reporting-figures-and-demo

### 目的

將實驗成果包裝成 final project 可交付成果。

### 啟動條件

Change 04 已完成並經使用者確認。

### 必須輸出

- final README
- final figures
- experiment summary
- PPT assets
- demo script
- final report outline
- known limitations

### 驗收標準

- GitHub repo 可被第三方依 README 重現主要流程
  - 圖表可直接放入 PPT
  - 10 分鐘 demo script 可完整說明：
    - problem
    - related work
    - baseline
    - DRL design
    - experiment
    - result
    - limitation
  - 所有 final artifacts 放在明確資料夾中
- 

## 16. 執行順序規定

嚴格執行以下順序：



1. 官方 OpenSpec 安裝與驗證
2. `openspec init --tools antigravity --profile core`
3. 驗證 `.agent/skills/` 與 `.agent/workflows/`
4. Change 01 : project-charter
5. 等待使用者確認
6. Change 02 : ns3-baseline-benchmark
7. 等待使用者確認
8. Change 03 : ns3-gym-environment
9. 等待使用者確認
10. Change 04 : dqn-mvp-agent
11. 等待使用者確認
12. Change 05 : reporting-figures-and-demo

禁止跳過 baseline 直接做 DQN。

禁止在 ns3-gym smoke test 前開始正式訓練。

禁止在 OpenSpec change 未完成前直接實作。

禁止一次處理多個 change，除非使用者明確允許。

禁止使用自製 OpenSpec 模擬流程。

## 17. 風險控管與降階策略

### Risk 1：官方 OpenSpec 安裝或 Antigravity integration 失敗

#### 等級

最高

#### 預防方式

- 使用官方 npm 套件
- 驗證 `openspec CLI`
- 使用 `openspec init --tools antigravity`
- 檢查 `.agent/skills/` 與 `.agent/workflows/`
- 使用 `openspec update --force`

#### 降階方案

沒有降階成仿製 OpenSpec 的選項。

若官方 OpenSpec 無法使用，必須停止並回報，等待使用者決策。

## Risk 2：ns3-gym 安裝失敗

### 等級

高

### 預防方式

- 固定 Linux / Ubuntu 環境
- 固定 ns-3 版本
- 先跑官方 example
- 安裝步驟寫入 README
- 保留完整錯誤 log

### 降階方案

若 ns3-gym 超過 3 次修復嘗試仍無法完成 reset / step smoke test，停止擴張並回報使用者，提出 fallback：

1. simplified Python Gym bottleneck simulator
2. 先完成 ns-3 baseline + DRL design proposal
3. 改以既有 ns3-gym example 進行最小改造

不得靜默改方向。

---

## Risk 3：BBR baseline 整合困難

### 等級

中高

### 預防方式

- 先完成 NewReno / CUBIC
- BBR 設為 strongly preferred，但不是 blocking dependency

### 降階方案

若 BBR 阻塞超過 1 個工作日，將 BBR 移至 optional / future work，並在 README 與 report 中說明原因。

---

## Risk 4：無法直接控制 cwnd

### 等級

高

## 預防方式

- 先確認 ns-3 / ns3-gym 可控接口
- 不假設一定能直接控制 TCP cwnd
- 在 design.md 中明確記錄控制層級

## 降階方案

若直接控制 cwnd 成本過高，改為：

1. application sending rate control
2. simplified rate limiter
3. cwnd-like abstract control signal

任何降階都必須更新 OpenSpec **design.md**，並等待使用者確認。

---

# Risk 5：Reward 設計導致 agent 學歪

## 等級

高

## 預防方式

- Reward 不得只看 throughput
- 必須加入 RTT / loss penalty
- 必須保留 reward component log
- 必須比較 utility，而不是只看單一 throughput

## 降階方案

若 reward 學習不穩，先固定權重並降低 scenario 複雜度；若仍不穩，改成 trade-off analysis，不宣稱 DRL 優於 baseline。

---

# Risk 6：DQN 表現不佳

## 等級

中高

## 預防方式

- action space 保持簡單
- scenario 先保持簡單
- 先用 random agent 做 smoke test
- 先追求完整 pipeline，不追求 SOTA

## 降階方案

若 DQN 沒有贏 baseline，不得偽造結果。改成：

- 分析 reward curve
- 分析 throughput / RTT / loss trade-off
- 說明 DRL 在哪些情境接近 baseline
- 將 PPO / reward shaping / multi-flow 放入 future work

## Risk 7：題目擴張到 IPFS / QUIC / Multi-agent

### 等級

最高

### 預防方式

- 在 project-charter 中寫入 non-goals
- 每個 change 都檢查是否違反 scope
- 所有擴張都要先問使用者

### 降階方案

若任務開始偏向 IPFS / QUIC / multi-agent，立即停止該 change，回到 project-charter 重審。

## 18. Commit / PR / 交付要求

每次完成一個 change，都必須交付：

- OpenSpec change folder
- 對應程式碼，若該 change 有實作任務
- README 更新
- validation command
- sample output
- logs
- figures，若該 change 有要求

- 已知問題清單
- 下一步建議

Commit message 建議格式：

```
change-01: add official openspec project charter
change-02: add ns-3 baseline benchmark for bottleneck topology
change-03: add ns3-gym environment smoke test
change-04: train initial DQN congestion-control agent
change-05: add reporting figures and demo assets
```

禁止提交無法說明用途的散亂程式碼。

## 19. README 最低要求

README 必須包含：

1. Project title
2. Research motivation
3. Scope and non-goals
4. Official OpenSpec setup proof
5. Toolchain
6. Installation
7. How to run baseline
8. How to run ns3-gym smoke test
9. How to train DQN
10. How to reproduce figures
11. Results summary
12. Known limitations
13. Future work

README 必須讓不參與對話的人也能理解並重現主要流程。

## 20. 最終成果標準

本專案完成時，至少應具備：

- 官方 OpenSpec 初始化紀錄
- `.agent/skills/` 與 `.agent/workflows/` 產物
- OpenSpec changes
- 可重現的 ns-3 baseline
- 可執行的 ns3-gym environment smoke test

- 可訓練的 DQN MVP
- baseline vs DRL comparison
- throughput / RTT / loss / utility 圖表
- README
- final report outline
- PPT assets
- demo script

若 DQN 結果不佳，只要實驗流程完整、baseline 清楚、設計合理、限制誠實，仍可作為 final project 成果。

## 21. 嚴格禁止事項

你不得：

- 自行改題目
- 自行加入 IPFS 實作
- 自行加入 QUIC 實作
- 自行改 Linux kernel TCP
- 自行改成 multi-agent RL
- 自行擴大 topology
- 自行跳過官方 OpenSpec CLI
- 自行手刻假 OpenSpec
- 自行模擬 OpenSpec workflow
- 自行跳過 baseline
- 自行跳過 smoke test
- 偽造實驗結果
- 在結果不好時宣稱全面優於 baseline
- 在未經使用者確認前新增大型依賴
- 將 optional 當作 MVP 必做
- 將 future work 寫成已完成成果

## 22. 開工第一步

你現在的第一個任務不是寫 baseline code，也不是寫 DQN code。

你必須先完成官方 OpenSpec setup：

```
Bash
npm install -g @fission-ai/openspec@latest
node -v
```

```
npm list -g @fission-ai/openspec --depth=0
which openspec || where openspec
openspec --version
openspec --help
openspec init --tools antigravity --profile core
openspec update --force
find . -maxdepth 4 -type f | sort
```

接著確認：

```
openspec/
.agent/skills/
.agent/workflows/
```

都已存在。

然後使用官方 OpenSpec workflow 建立第一個 change：

```
/opsx:propose create project charter for DRL bottleneck congestion-control final project
```

若 Antigravity 實際顯示的命令格式不是 `/opsx:propose`，請讀取 `.agent/workflows/` 內官方產生的 workflow 檔案，使用其中定義的官方格式。

第一個 change 只能建立並完善：

```
openspec/changes/<official-change-name>/
├── proposal.md
├── design.md
├── tasks.md
└── specs/project-charter/spec.md
```

內容必須包含：

- Project title
- Research goal
- Research background
- Scope
- Non-goals
- MVP
- Baselines
- Metrics
- Toolchain
- Official OpenSpec setup proof
- Risk register
- Acceptance criteria
- Change sequence
- Open questions
- What must not be changed without approval

完成後先停止，等待 spec owner 確認。

## 23. 每次回覆格式

每次完成任務後，你必須用以下格式回覆：

```
本次完成項目
- ...

官方 OpenSpec 使用證據
- 已執行指令：
- 產生的 openspec 檔案：
- 產生的 .agent 檔案：

產出檔案
- ...

驗證方式
- ...

已知問題
- ...

是否需要使用者確認
- 是 / 否

下一步建議
- ...
```

如果遇到阻塞，請用以下格式回報：

```
阻塞點
...

已嘗試方法
...

錯誤訊息 / log 摘要
...

可能原因
...

建議降階方案
...

需要使用者決策
...
```

## 24. 最終提醒

本專案的核心精神是：

- Official OpenSpec only
- No fake OpenSpec
- Spec first
- Baseline first
- Smoke test before training



- MVP before optimization
- Honest results over inflated claims
- Reproducibility over complexity
- Scope control over feature expansion

現在請只開始執行：

**Official OpenSpec setup + Change 01: project-charter**

不要寫 baseline code。

不要寫 DQN code。

不要安裝與本階段無關的大型依賴。

不要改題目。

不要手刻 OpenSpec。

不要模擬 OpenSpec。

先用官方 OpenSpec 套件完成 setup，並產出 Change 01 的完整 proposal / design / tasks / spec。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/1f5a9cf7e57a57664c827488ee2e5e0e3a85aea2>

這是 Antigravity 做完「Phase 0：定案總結」的成果 請去看 git diff

幫我驗收 Antigravity 做的可以嗎 有滿分淺力嗎

尤其 proposal.md、design.md、spec.md 的內容是否與研究方向吻合（題目、MDP 定義、Non-goals 清單）

顯示更多顯示較少

可以驗收，而且我有看這個 commit 的實際內容。

我的結論先講在前面：

**Antigravity 這次 Change 01 / Phase 0 的成果方向是對的，有高分潛力，但目前我不建議直接放行進入 Change 02。**

它已經完成了「題目凍結、MVP 範圍、MDP 初版、工具鏈、baseline、non-goals」這幾個最重要的方向控制，**proposal.md、design.md、spec.md 大方向都吻合我們原本規劃**。但有幾個地方要修掉，尤其是 **Node.js / 官方 OpenSpec requirement、topology non-goal 表述、cwnd fallback、tasks 未全勾完**。修完後，我會評為「可以進入 Change 02」。

## 一、總評

## 驗收結論

項目	結論
研究方向是否吻合	吻合
題目是否凍結正確	正確
MDP 定義是否合理	基本合理，但 <code>cwnd fallback</code> 要補強
Non-goals 是否完整	大致完整，但 <code>topology</code> 表述需修正
是否真的使用 OpenSpec	看起來有用官方 OpenSpec 產物，但 <code>setup proof</code> 要補更完整
是否可進 Change 02	暫不建議，先修 4 個問題
滿分潛力	有，但目前是 85-88 分等級；修完可到 92-95 分以上

這次 commit message 是 `change-01: add official openspec project charter`，而且 commit diff 裡確實包含 OpenSpec 產生的 skill / workflow 內容，以及 project-charter 相關 artifacts。

[change-01: add official openspe...](#)

## 二、proposal.md 驗收

### 2.1 題目與方向：通過

proposal.md 的 Why 寫得正確。它明確說明本期末專題要完成一套可重現、可驗收、可展示的 DRL research pipeline，主題是「以 DRL 進行單一瓶頸鏈路壅塞控制與吞吐量最佳化」，並且說明目前需要 project charter 作為唯一真相文件。這和我們 Phase 0 的目的完全一致。

[proposal](#)

### 2.2 Change 範圍：通過

proposal.md 明確列出要建立 `openspec/changes/project-charter/`，並凍結研究題目、研究目標、MDP 初版、工具鏈、實驗情境，也定義後續五個 OpenSpec changes 的順序。這符合我們要求的「Phase 0 一槌定音」定位。

[proposal](#)

### 2.3 工具鏈與 Non-goals：通過

proposal.md 的 Impact 區明確界定 ns-3、ns3-gym、Stable-Baselines3、DQN 為核心工具鏈，並排除 IPFS、QUIC、multi-agent RL、Linux kernel TCP、large-scale topology 等非 MVP 範疇。這點非常重要，因為它避免 Antigravity 後續把題目做爆。

[proposal](#)

### 2.4 proposal.md 小問題

proposal.md 目前只有 31 行，作為 OpenSpec change proposal 是可以的，但如果你要把它拿去當「期末提案材料的上游文件」，它還可以再補兩句：

1. 為何本題和網路吞吐量 / IPFS 背景有知識連續性
2. 為何本題不直接做 IPFS，而先從 bottleneck congestion control 收斂

這不是阻塞問題，但補上會更像滿分規格。

## 三、design.md 驗收

### 3.1 角色與專案狀態：通過

design.md 明確定義使用者是 Spec Owner / Project Owner，Antigravity 是 Implementation Agent，而且寫明目前尚無 baseline code、RL environment 或訓練腳本。這符合你「我負責規劃，Antigravity 負責實作」的角色設定。

design

### 3.2 Goals / Non-Goals：大致通過

Goals 寫得很準，包含題目、thesis statement、MDP 初版、MVP 必做清單、risk register、acceptance criteria、工具鏈、metrics。這些就是 Phase 0 該凍結的內容。

design

Non-goals 也大致正確，明確排除 IPFS、QUIC、Linux kernel TCP、multi-agent、large topology、多路徑、分散式節點、production-grade TCP、real Internet deployment、Pantheon 必裝依賴，以及不承諾 DRL 全面勝過所有 TCP。

design

### 3.3 重要問題：large topology 表述要修

這句有風險：

不做大型網路拓樸 (> 2 nodes with 1 bottleneck link)

這個表述不精準。單一瓶颈鏈路不一定只有 2 個 nodes。

在 ns-3 裡，常見 bottleneck topology 可能是：

```
sender → router_1 → router_2 → receiver
```

或 dumbbell topology：

```
sender(s) → router_1 == bottleneck == router_2 → receiver(s)
```

所以「> 2 nodes」可能會誤傷合理的單一 bottleneck 實作。建議改成：

不做大型網路拓樸；MVP 僅限單一 bottleneck path，不做多 bottleneck、多路徑、多 sender/receiver 群組或大規模 topology。

這樣比較符合研究方向，也不會阻礙 Change 02 做合理 ns-3 topology。

## 3.4 工具鏈決策：通過

D1 凍結為 ns-3 + ns3-gym + Stable-Baselines3 + DQN，並排除 GNS3 / real testbed、SUMO 等不相關工具，方向正確。

design

官方資料也支持這個決策：ns3-gym 官方說明它整合 OpenAI Gym 與 ns-3，用於 networking research 的 RL 使用；ns-3 TCP model 官方則提供 NewReno、CUBIC、BBR 等 TCP congestion control 基礎。 [GitHub+3](#)

## 3.5 DQN 決策：通過

D2 決定 MVP 使用 DQN、離散 action space，PPO / SAC 放到 future work，這是正確決策。Stable-Baselines3 的 DQN 文件也明確顯示 DQN 支援 **Discrete** action space，不支援 **Box** action space；這和我們規劃的 {decrease, keep, increase} 離散控制完全對齊。

design

 stable-

[baselines3.readthedocs.io](https://baselines3.readthedocs.io)

## 3.6 MDP 定義：基本通過，但要補 fallback

design.md 的 D3 寫：

```
observation = {throughput, RTT, loss_rate, cwnd_signal}
action = {decrease, keep, increase}
reward = $\alpha \cdot \text{throughput} - \beta \cdot \text{RTT} - \gamma \cdot \text{loss}$
```

這和我們原本規劃一致，reward 也正確避免只追 throughput。

design

但這裡要補強一點：cwnd\_signal 目前還是未確認可直接取得 / 控制。ns3-gym 的 RL-TCP example 確實提到已提供介面可監控 TCP instance parameters，並控制 Congestion Window 與 Slow Start Threshold，但 Antigravity 後續仍要在 Change 03 驗證實際可行性。 [GitHub](#)

所以 design.md 應該補一句：

若 Change 03 驗證無法穩定取得或控制 cwnd，MVP observation / action 可降階為 application sending rate 或 cwnd-like abstract control signal，且需更新 design.md 並回報 Spec Owner。

目前 design.md 有提到「直接控制 cwnd vs sending rate 待 Change 03 確認」，這很好；但 spec.md 也要同步補進 fallback，不然 specification 會太硬。

design

## 3.7 Baseline 決策：通過

D4 凍結 NewReno + CUBIC 必做，BBR strongly preferred 但非 blocking。這是正確設計。ns-3 官方文件明確有 NewReno、CUBIC、BBR，而且 BBR 是根據 bottleneck bandwidth 與 RTT 估計控制 sending rate，正好可作為和 DRL rate-control 類方法比較的高價值 baseline。

design

 nsnam.org+2

## 3.8 Episode 與 scenarios：通過但需標註初始值

60s episode、500ms decision interval 作為初始值可以接受，但要保持「初始假設」定位，不要讓它變成不可調。design.md 已寫「實際值待 smoke test 確認後可調整」，這是正確的。

design

## 3.9 Risk Register：通過

Risk Register 有把 ns3-gym 安裝、BBR、cwnd、reward、DQN、IPFS/QUIC 擴張、OpenSpec CLI、Node.js 版本都列出來。尤其 Node.js 版本風險寫得很好，因為 OpenSpec 官方 Quick Start 明確要求 Node.js 20.19.0 or higher，而 README 裡記錄的是 Node v20.11.1。這是需要修的點，不是可忽略的小事。

design

 GitHub

# 四、spec.md 驗收

## 4.1 題目與 thesis：通過

spec.md 的第一個 requirement 明確把中文題目、英文題目、GitHub title、thesis statement 凍結，且寫明未經 Spec Owner 核准不得更動題目、thesis 或研究方向。這是非常好的規格寫法。

spec

## 4.2 Change sequence：通過

spec.md 清楚定義五個 OpenSpec changes：

1. project-charter
2. ns3-baseline-benchmark
3. ns3-gym-environment
4. dqn-mvp-agent
5. reporting-figures-and-demo

並且規定新 change 必須在前一個 change 完成且經 Spec Owner 確認後才能開始。這正是你要的「防止 Antigravity 偏航」機制。

spec

## 4.3 Toolchain：大致通過，但版本約束要修

spec.md 凍結 ns-3、ns3-gym、Stable-Baselines3、DQN、Python、OpenSpec，整體方向正確。

spec

但這裡有兩個要修的細節：

### 問題 1：OpenSpec Node 版本不合官方要求

README 寫 Node v20.11.1，OpenSpec v1.4.1。

README

但 OpenSpec 官方 README 明確要求 **Node.js 20.19.0 or higher**，並提供 `npm install -g @fission-ai/openspec@latest` 與 `openspec init` 流程。[GitHub](#)

所以這裡不能只說「WARN 但功能正常」。建議修為：

Current environment has Node.js v20.11.1, which is below OpenSpec official requirement 20.19.0+. Change 01 may be reviewed, but before Change 02 begins, Node.js MUST be upgraded or an explicit Spec Owner waiver must be recorded.

### 問題 2：SB3 >= 1.8.0 可接受，但最好不要鎖太舊

Stable-Baselines3 目前文件版本已到 2.9.0a2；spec.md 寫 `>= 1.8.0` 不算錯，但建議 Change 04 時改成「使

 stable-

用可和實際 Python / Gymnasium 環境相容的穩定版本」，避免將來 package 相依卡死。[baselines3.readthedocs.io](https://baselines3.readthedocs.io)

## 4.4 MDP Formulation：基本通過，但要修 cwnd\_signal 的硬約束

spec.md 寫：

```
Observation: [throughput_norm, rtt_norm, loss_rate, cwnd_signal_norm]
Action: Discrete(3)
Reward: $\alpha \cdot \text{throughput} - \beta \cdot \text{RTT} - \gamma \cdot \text{loss}$
Episode length: 60s
Decision interval: 500ms
```

這是方向正確的 MDP 初版。

spec

但 `cwnd_signal_norm` 在 spec.md 裡看起來太像必須取得的硬 requirement。由於 design.md 已經承認 cwnd 控制層級待 Change 03 確認，spec.md 應補成：

```
Observation initial candidate:
[throughput_norm, rtt_norm, loss_rate, cwnd_signal_norm OR sending_rate_signal_norm]
```

並新增 scenario：

若 cwnd 無法穩定取得或控制，Change 03 MUST propose fallback to sending-rate control and update design.md before implementation continues.

這樣會更符合工程現實，也更容易讓 Antigravity 執行。

## 4.5 Baseline：通過

spec.md 明確要求 baseline 必須在 RL environment 或 DRL training 前完成，並定義 NewReno、CUBIC 必做，BBR strongly preferred 但一個工作日內失敗就移到 optional / future work。這非常好。

spec

## 4.6 Metrics：通過，但 utility formula 要前置

spec.md 已經要求 Average throughput、Average RTT、Packet loss rate、Utility score、Reward curve、Baseline comparison table，並明確要求 utility score formula 必須在比較前定義，而且 baseline 與 DRL 必須一致套用。這是正確且很重要的驗收條件。

spec

建議加一條：

Reward components and utility components MUST be logged separately.

因為這樣後面 DQN 結果不好時，才能解釋到底是 throughput 變高、RTT 變差，還是 loss 造成 penalty。

## 4.7 Experiment scenarios：通過但數值需留彈性

Scenario A / B 設定 10 Mbps、20ms / 100ms RTT，並要求固定 seed。這對 MVP 很好。

spec

但建議加一句：

Scenario values are initial MVP defaults and may be tuned in Change 02 design.md if ns-3 implementation requires different link-delay or bandwidth parameterization.

這能避免 Antigravity 被 spec 卡住。

## 4.8 Final deliverables：通過

spec.md 列出 11 個 required deliverables，包含 GitHub README、baseline CSV/figures、ns3-gym smoke test、DQN reward curve、DRL vs baseline comparison、topology diagram、MDP diagram、report outline、PPT assets、demo script。這很完整，足以支撐期末展示。

spec

## 4.9 Non-goals enforcement：通過但 topology 那句要修

Non-goals 很完整，包括 IPFS、Bitswap、DHT、libp2p、blockchain network protocol、QUIC、kernel TCP、real Internet deployment、multi-agent、large topology、multi-path、distributed node systems、Pantheon mandatory dependency、不能宣稱 DRL universal outperform。

spec

但同樣，「Large-scale topology (> sender + bottleneck + receiver)」要改成「多 bottleneck、多 path、多 sender/receiver 群組或大規模 topology」，不要限制合理的 router node 數。

## 五、tasks.md 驗收

### 5.1 目前不能算 fully complete

tasks.md 裡還有兩個未完成項目：

- 3.5 Create proposal/ directory...
- 5.6 Present completed charter to Spec Owner for confirmation before starting Change 02

這代表 Change 01 還沒有完全完成。

tasks

tasks

其中 5.6 正是現在這個驗收動作，所以可以合理保留未勾。但 3.5 要請 Antigravity 處理清楚：如果 proposal directory 已經存在，就應該把 checkbox 勾上並說明；如果不存在，就建立。

### 5.2 OpenSpec 使用證據：基本通過，但要補官方 init proof

tasks.md 已勾選 OpenSpec CLI installed、v1.4.1、.agent/skills、.agent/workflows、openspec new change "project-charter" 等項目。

tasks

README 也列出 OpenSpec v1.4.1、.agent/skills/openspec-\* 與 .agent/workflows/opsx-\* 產物。這能初步證明它不是手刻假 OpenSpec。

README

但我建議補上更硬的 proof：

Bash

```
openspec --version
openspec status --change "project-charter" --json
openspec instructions proposal --change "project-charter" --json
openspec instructions design --change "project-charter" --json
openspec instructions specs --change "project-charter" --json
openspec instructions tasks --change "project-charter" --json
```



並把輸出摘要放進 README 或 `docs/openspec_setup_proof.md`。

## 六、README 額外觀察

README 已經有研究動機、scope、non-goals、OpenSpec setup proof、toolchain、run baseline / smoke test / DQN / figures 的 TODO、結果表、change roadmap、known limitations、future work。整體作為 repo 首頁很不錯。

README

README README

但有一個嚴重小雷：

README 最上方放了一個 YouTube 影片連結：

```
https://www.youtube.com/watch?v=3cu1WZz_k8M
```

目前看起來像 placeholder 或不明影片，不是你的期末簡報影片。

README

這個要改掉。建議：

- Phase 0 / Change 01 階段先移除 YouTube block
- 或改成 `TODO: add final proposal video link`
- 不要放不明影片，否則老師或助教點進去會扣印象分

## 七、是否有滿分潛力？

### 我的判斷：有，但要修完再放行

目前成果是：

**Phase 0 / Change 01：85-88 分**

修完我下面列的問題後，可以到：

**Phase 0 / Change 01：92-95 分**

之後如果 Change 02-05 也按照這個 spec 做出：

- baseline 可重現
- ns3-gym smoke test 可跑
- DQN 能訓練
- 圖表完整
- 結果誠實分析

那整個 final project 有滿分潛力。

它的高分基礎在於：

1. 研究方向和 DRL 課程吻合
2. 題目和你的網路 / IPFS 背景有連續性
3. OpenSpec 工作流有明確落地
4. MVP 邊界清楚
5. Non-goals 足夠完整
6. baseline-first 的策略正確
7. reward 不只看 throughput，避免研究設計過於幼稚

## 八、我要求 Antigravity 修正的清單

請你不要讓它直接進入 Change 02。

先請 Antigravity 做一個 **Change 01 revision**。

### 必修 1：修正 Node.js / OpenSpec official requirement

目前 README 寫 Node v20.11.1，但 OpenSpec 官方要求 Node.js 20.19.0+。

[README](#)



請修成：

Current Node.js version is below official OpenSpec requirement. Before Change 02, either upgrade Node.js to 20.19.0+ or record a Spec Owner-approved waiver.

### 必修 2：修正 topology non-goal 表述

把：

不做大型網路拓模 (> 2 nodes with 1 bottleneck link)

改成：

不做大型網路拓模；MVP 僅限 single bottleneck path，不做多 bottleneck、多 path、多 sender/receiver 群組或大規模 topology。

spec.md 的 forbidden item 也要同步改，不要用 > sender + bottleneck + receiver 卡死合理 ns-3 router topology。

### 必修 3：補上 cwnd fallback 到 spec.md

spec.md 目前把 cwnd\_signal\_norm 寫得太硬。

spec

請改成：

```
Observation initial candidate:
[throughput_norm, rtt_norm, loss_rate, cwnd_signal_norm OR sending_rate_signal_norm]
```

並新增：

```
IF cwnd cannot be reliably observed or controlled through ns3-gym,
THEN Change 03 MUST propose a fallback to sending-rate control or cwnd-like abstract control
signal,
AND update design.md before implementation continues.
```

## 必修 4：移除 README 不明 YouTube link

README 上方的 YouTube block 目前不是本專題正式影片。

README

請改成：

```
<!-- TODO: Add final proposal video link after recording -->
```

或整段先移除。

## 必修 5：完成 tasks.md 未勾項目

tasks.md 還有 3.5 和 5.6 未完成。

tasks

tasks

5.6 可以在你確認後勾。

3.5 若已存在就勾上；若不存在就建立。

## 九、給你可以直接貼給 Antigravity 的驗收意見

本次 Change 01: project-charter 已完成初步驗收，整體方向正確，proposal.md、design.md、spec.md 與研究主題「以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化」基本吻合。

但目前尚不可進入 Change 02。請先完成以下 revision：

- 修正 Node.js / OpenSpec official requirement：
  - README 顯示 Node v20.11.1，但 OpenSpec 官方要求 Node.js 20.19.0+。
  - 請在 README 與 design.md 中明確標註此風險。
  - Change 02 前必須升級 Node.js 至 20.19.0+，或取得 Spec Owner 明確 waiver。
- 修正 topology non-goal：
  - 不要寫「> 2 nodes with 1 bottleneck link」或「> sender + bottleneck + receiver」。
  - 改成：MVP 僅限 single bottleneck path，不做多 bottleneck、多 path、多 sender/receiver 群組或大規模 topology。

- 允許合理 ns-3 router-based bottleneck topology。

### 3. 補上 cwnd fallback：

- spec.md 不可硬性要求一定有 cwnd\_signal\_norm。
- 改成：cwnd\_signal\_norm OR sending\_rate\_signal\_norm。
- 新增規則：若 Change 03 驗證無法穩定取得或控制 cwnd，必須提出 sending-rate control 或 cwnd-like abstract control signal fallback，並更新 design.md，等待 Spec Owner 確認。

### 4. 移除 README 上方不明 YouTube link：

- 該影片目前不是本專題正式影片。
- 請先移除或改為 TODO placeholder。

### 5. 完成 tasks.md 未勾項目：

- 3.5 若 proposal/ directory 已存在，請勾上並說明；若不存在，請建立。
- 5.6 等 Spec Owner 確認後再勾。

完成以上 revision 後，請回報：

- 修改過的檔案
- 修改摘要
- 新的 git diff
- openspec status --change "project-charter" --json 摘要
- 是否已準備進入 Change 02

在 revision 完成前，不得開始 Change 02：ns3-baseline-benchmark。

## 十、最終驗收結論

可以，但目前是「條件式通過」。

我不會說它做得不好；相反，Antigravity 這版已經比很多 AI agent 做出來的規格清楚很多，特別是題目、MDP、baseline、non-goals、change sequence 都有抓住。

但如果你目標是滿分，不要急著進 Change 02。

請先讓它修掉上面 5 點。修完後，這個 Change 01 就可以作為整學期專題的正式 source of truth。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/dea78ef0eb3f19705a18b304b6b3b3b5dc7cebe5>

請看此 github repo

我已經請 Antigravity 修完以上 5 點

請幫我驗收是否完善

並判斷專案是否可推進到 Change 02

顯示更多顯示較少

可以，我已經看過這個 commit。

**結論：**這版已經可以視為 Change 01 條件式通過，專案可以推進到 Change 02，但我建議先做 3 個很小的清理，然後正式允許 Antigravity 開始 Change 02 implementation。

這次 commit message 已明確標示：Change 01 已修正 Node.js 風險、topology wording、cwnd fallback、README cleanup；同時也建立了 Change 02：ns3-baseline-benchmark 的 proposal artifacts。

[change-01: revisions \(Node.js r...](#)

另外目前 GitHub 沒有 workflow runs，所以這次驗收是**文件與規格驗收**，不是 CI / 實作驗收。

## 一、5 點修正驗收結果

修正項	驗收結果	判斷
Node.js / OpenSpec requirement	已補風險說明	通過，但 implementation 前建議升級或正式 waiver
Topology wording	主規格已修正，README 還有一處舊字句	小修，不阻塞 Change 02
cwnd fallback	已補到 spec / scope	通過
README YouTube link	已移除，改成 TODO	通過
tasks.md 未勾項目	大多完成，只剩 final confirmation / archive 類任務	通過，需你確認後勾 12.6

## 二、逐項驗收

### 1. Node.js / OpenSpec requirement

這次有補上 Node.js 版本風險。README 已寫明目前 Node.js 是 **v20.11.1**，低於 OpenSpec 官方要求的 **v20.19.0+**，並建議 Change 02 implementation 前升級或由 Spec Owner 給出 waiver。

[README](#)

design.md 也把這件事放進 Technical Direction 與 Open Questions，說明目前功能正常，但 Change 02 implementation 前建議升級或取得 waiver。

[design](#)

[design](#)

這是正確修法。OpenSpec 官方 Quick Start 也明確要求 Node.js 20.19.0 或更高，並以 `npm install -g @fission-ai/openspec@latest` 安裝、`openspec init` 初始化。[GitHub](#)

**判斷：通過。**

但我建議你在允許 Antigravity 開始 Change 02 implementation 前，二選一：

- A. 升級 Node.js 到 20.19.0+
- B. 你明確回覆：Spec Owner 同意暫時 waiver，Change 02 可先執行，但若 OpenSpec CLI 出現問題立即停止

### 2. Topology wording

這點主規格已經修得很好。design.md 現在明確寫：MVP 僅限 single bottleneck path，但**合理的 ns-3 router-based bottleneck topology 是允許的**，禁止的是 multi-bottleneck、multi-path、multiple sender/receiver groups、large-scale topology。

design

scope.md 也補得更完整，明確說合理的 ns-3 router-based bottleneck topology 是允許的，中間 router 可以是瓶頸；禁止的是多瓶頸、多路徑、large fan-out topology。

scope

Change 02 design 也已採用 **sender → router → receiver** 的 router-based topology，這是合理的 ns-3 bottleneck 實作方式。

design

**唯一小問題：README 還留著舊句子。**

README 的 Out of Scope 仍寫：

Large-scale network topology (> sender + bottleneck + receiver)

這句不夠精準，容易和 design / scope 的 router-based topology 容許規則衝突。

README

**判斷：主規格通過，README 小修。**

這不阻塞 Change 02，但建議先修成：

✗ Large-scale network topology beyond single bottleneck path  
Reasonable router-based bottleneck topology is allowed; multi-bottleneck, multi-path, or multiple sender/receiver groups are out of scope.

### 3. cwnd fallback

這點已經修得很到位。project-charter spec 現在不再硬寫 `cwnd_signal_norm`，而是寫成：

`[throughput_norm, rtt_norm, loss_rate, cwnd_or_rate_signal_norm]`

而且明確定義 fallback rule：優先使用 `cwnd_signal_norm`；若 Change 03 smoke test 無法穩定取得或控制 cwnd，必須改為 `sending_rate_signal_norm` 或等效 congestion-control abstraction，並在 Change 03 design.md 記錄 Decision Record、等待 Spec Owner 確認後才可繼續。

spec

scope.md 也同步補上 cwnd fallback rule。

scope

這點非常重要，因為 ns3-gym 可以做 RL networking research，但實際 cwnd 讀寫是否穩定，必須到 Change 03 smoke test 才能確認。ns3-gym 官方定位就是整合 ns-3 與 Gym，支援 reinforcement learning in networking research；所以現在把 cwnd 控制列成「優先候選 + fallback」是最穩的規格寫法。  [GitHub](#)

**判斷：通過。**

### 4. README YouTube link

已修正。README 上方不再嵌入不明 YouTube 影片，而是改成：

Demo Video: TODO — 正式 demo video 將在 Change 05 完成後補上。

[README](#)

判斷：通過。

## 5. tasks.md 未勾項目

Change 01 tasks 現在大多已完成，包含官方 OpenSpec setup、研究基礎、scope、baseline、metrics、MVP、risk register、document review、scope compliance check，以及你前面要求的 5 項 revision。

[tasks](#)

[tasks](#)

目前剩下：

- **11.8 Archive change-01**：這本來就應該等整個專案或至少該 change 完整驗收後再處理。

[tasks](#)

- **12.6 Spec owner final confirmation of revisions before entering Change 02 implementation**：這就是你現在叫我做的驗收。

[tasks](#)

判斷：通過。

等你看完我這則驗收後，可以請 Antigravity 把 12.6 勾起來。

## 三、proposal.md / design.md / spec.md 是否吻合研究方向？

### proposal.md：吻合

proposal.md 現在比上一版完整很多。它清楚說明 project charter 的必要性：避免需求散落在聊天紀錄、避免 Antigravity 偏向 IPFS / QUIC / multi-agent、避免沒有 baseline 就直接跳去做 DQN，也避免期末驗收無法確認做了什麼。

[proposal](#)

它也明確列出本 change 不寫 code、不建立 ns-3 simulation、不做 ns3-gym、不訓練 DQN、不實作 IPFS / QUIC、不修改 kernel、不宣稱已有結果。這和 Phase 0 / Change 01 的定位完全一致。

[proposal](#)

驗收：通過。

### design.md：吻合

design.md 現在已經從「程式設計」轉成「project governance」，這是正確方向。它明確說本 design 的目標是專案治理，不包含 ns-3 參數、DQN 超參數或程式實作決策；這些會留給下游 change。

design

它也清楚凍結技術方向：ns-3、ns3-gym、SB3、DQN、NewReno / CUBIC、BBR non-blocking、throughput / RTT / packet loss / utility / reward curve 等。

design

它把 official OpenSpec package 也寫進 Decision Record，並明確禁止 fake OpenSpec / simulated OpenSpec，這正好回應你先前遇到的問題。

design

驗收：通過。

## spec.md：吻合

project-charter spec.md 已經把「single source of truth」寫得很清楚，規定所有後續 OpenSpec changes 都必須以本 charter 為上游規格依據。

spec

題目、mission、research target、MDP 初版、success definition、OpenSpec governance、downstream change map 都已完整對齊。尤其 MDP 定義現在包含 cwnd / rate fallback，非常關鍵。

spec

success definition 也正確：它要求 baseline、ns3-gym smoke test、DQN reward curve、DRL vs baseline comparison、GitHub 可重現、10 分鐘 demo script；但不要求 DRL 全面打敗所有 baseline，也不要求真實網路部署、IPFS、QUIC、production-grade TCP。

spec

驗收：通過。

## 四、Change 02 是否可以推進？

### 可以，但我要把話講準確

可以推進到 **Change 02 implementation**。

因為 Change 02 proposal / design / tasks / spec 都已建立，而且方向正確。

Change 02 proposal 明確說：在訓練 DRL agent 前，必須先建立可重現 TCP baseline benchmark；本 change 只做 ns-3 single bottleneck、NewReno / CUBIC / optional BBR、CSV log、baseline figures，不含 RL code。

proposal

Change 02 design 也明確定義：使用 router-based bottleneck topology、C++ script 優先、FlowMonitor 收 metrics、BBR non-blocking、YAML config、Python analysis scripts。

design



Change 02 tasks 也切得夠細：先檢查 ns-3 環境，再建 scenario config，再建 topology script，再跑 TCP baseline，再 parse CSV，再畫圖，再更新 README。這個順序是合理的。

tasks

tasks tasks

ns-3 官方 TCP model 文件也支持這條路線：ns-3 的 TCP model 支援多種 TCP congestion control 實作；這使 NewReno、CUBIC、BBR 作為 baseline 是合理的。[nsnam.org](https://nsnam.org)

## 五、進入 Change 02 前，請先做 3 個微修

這 3 個不是大問題，但我建議先補掉，避免之後實作時吵架。

### 微修 1：README 的 topology non-goal 舊句要改

README 仍保留：

**Large-scale network topology (> sender + bottleneck + receiver)**

請改成前面提到的版本。這是**文件一致性小修**，不影響主規格，但 README 是老師/助教最可能先看到的地方。

README

### 微修 2：Scenario A/B 的 RTT 與 link\_delay 要標清楚

scope.md 寫：

- Scenario A：RTT 20ms
- Scenario B：RTT 100ms

scope

但 Change 02 tasks 寫：

- scenario\_a link\_delay: "10ms"
- scenario\_b link\_delay: "50ms"

tasks

這可能是正確的，因為 link\_delay 可能是單向 propagation delay，所以 RTT 約為 2 倍。但它需要寫清楚。請 Antigravity 在 Change 02 design.md 或 config comments 補一句：

**link\_delay denotes one-way propagation delay on the bottleneck link; expected RTT is approximately  $2 \times \text{link\_delay}$  plus access-link delay.**

否則之後你會看到「為什麼 scope 說 RTT 20ms，config 卻寫 10ms」的混淆。

## 微修 3：Node.js waiver 或升級要明確決策

目前已記錄 Node.js 版本低於 OpenSpec 官方要求，但尚未真正升級。README 與 design 都寫得清楚，這是可以接受的。

[README](#)

但進 Change 02 implementation 前，請你直接下其中一個決策：

我批准暫時 waiver：Change 02 implementation 可在 Node.js v20.11.1 下進行；若 OpenSpec CLI 出現任何問題，立即停止並升級至 20.19.0+。

或：

請先升級 Node.js 到 20.19.0+，再開始 Change 02 implementation。

我的建議是：如果 Antigravity 已經能正常跑 OpenSpec，先給 waiver，不要讓 Node 卡住 baseline；但若中途 OpenSpec 指令異常，立即升級。

## 六、最終判斷

### 是否完善？

已經接近完善。

Change 01 的 proposal.md、design.md、spec.md 現在都和研究方向吻合，也有把 MDP、Non-goals、OpenSpec governance、cwnd fallback、baseline-first 策略寫清楚。

### 是否有滿分潛力？

有。

現在的專案架構已經從「AI 生成一堆文件」提升到「可治理、可驗收、可逐 change 推進」的等級。若 Change 02 能真的跑出 NewReno / CUBIC baseline CSV 與三張圖，整個專題的可信度會大幅上升。

### 是否可推進到 Change 02？

可以。

我建議你正式放行：

Change 01 revisions 已通過驗收。Spec Owner 確認專案方向、scope、MDP 初版、Non-goals、OpenSpec governance 均符合要求。允許進入 Change 02：ns3-baseline-benchmark implementation。

進入 implementation 前，請先完成以下 3 個微修：

1. 修正 README topology non-goal 舊句。
2. 在 Change 02 design/config 補充 link\_delay 是 one-way delay，預期 RTT 約為  $2 \times \text{link\_delay}$ 。
3. 由 Spec Owner 明確決定 Node.js 20.11.1 是否暫時 waiver；若無 waiver，請先升級至 20.19.0+。

完成後可開始 Change 02 tasks，但仍不得開始 ns3-gym、DQN、PPO 或任何 RL 程式碼。

這樣推進是安全的。